



DomOS

Agentic UI SDK

DomOS

DOM Operating System

CAHIER DES CHARGES

@domos/browser

SDK Universel — Sites sans Framework Frontend

Laravel · PHP · Shopify · WordPress · HTML statique

Version 1.0 — Mars 2026

Projet	DomOS — @domos/browser
Type de document	CdC Fonctionnel + Technique complet
Complément de	CdC Technique général (Doc 2)
Organisation	Futur4Tech
Responsable	Kouakou Boris (CEO)
Date	Mars 2026
Stack build	Preact + esbuild → CDN (IIFE) + ESM
Statut	En conception — roadmap v1.0

Table des matières

Table des matières	3
1. Vision et problème résolu	5
1.1 Le problème — DomOS sans framework frontend	5
1.2 La solution — @domos/browser	5
1.3 Plateformes cibles prioritaires	5
2. Acteurs et scénarios d'utilisation	7
2.1 Acteurs	7
2.2 CU-B01 — Intégration CDN sur une page HTML statique	7
Exemple d'intégration HTML statique	7
2.3 CU-B02 — Intégration Shopify (Liquid)	8
Exemple d'intégration Shopify Liquid	8
2.4 CU-B03 — Intégration Laravel (Blade)	9
Exemple d'intégration Laravel Blade	9
2.5 CU-B04 — Persistance de session lors d'un rechargement de page	10
Flux de persistance	11
2.6 CU-B05 — Auto-discovery des tools via attributs HTML	11
Exemple WordPress — déclaration 100% HTML	11
3. Fonctionnalités	13
3.1 Vue d'ensemble des fonctionnalités	13
3.2 API publique de @domos/browser	14
3.2.1 DomOS.init(config)	14
3.2.2 DomOS.registerTool(name, definition)	14
3.2.3 Autres méthodes de l'API publique	15
4. Auto-discovery — Attributs data-domos-*	16
4.1 Principe	16
4.2 Attributs disponibles	16
4.3 Actions DOM prédéfinies	16
5. Session Persistence — Architecture complète	18
5.1 Le problème des MPA (Multi-Page Applications)	18
5.2 Architecture de persistance — 3 niveaux combinés	18
5.3 Flux de reprise de session (Session Resume)	18
5.4 Configuration de la persistance	19
5.5 Intégration DomosAgent — Mémoire cross-session	19
6. Architecture technique de @domos/browser	21
6.1 Stack de build	21
6.2 Formats de distribution	21
6.3 Structure interne du SDK	21
6.4 Injection du Widget dans la page hôte	22

6.5 Compatibilité navigateurs	22
7. Guides d'intégration par plateforme.....	24
7.1 Guide Shopify	24
7.1.1 Injection dans theme.liquid.....	24
7.1.2 Tools globaux Shopify (toutes pages)	24
7.2 Guide WordPress	25
7.2.1 Enqueue via functions.php	25
7.3 Guide HTML statique (GitHub Pages / Netlify).....	25
Intégration minimale — 5 lignes	25
8. Cas d'erreur et comportements attendus.....	26
9. Exigences	27
9.1 Exigences fonctionnelles	27
9.2 Exigences techniques.....	27
10. Roadmap et annexes	29
10.1 Roadmap @domos/browser	29
10.2 Comparaison @domos/browser vs SDKs framework	29
10.3 Références	30

1. Vision et problème résolu

1.1 Le problème — DomOS sans framework frontend

DomOS (DOM Operating System) supporte aujourd'hui les frameworks frontend modernes — React, Vue, Svelte, Flutter — via des SDKs dédiés utilisant leurs systèmes de composants et cycles de vie. Cependant, une large partie du web ne fonctionne pas avec ces frameworks :

- Les sites générés côté serveur avec Laravel, PHP pur, Ruby on Rails ou Django.
- Les boutiques Shopify dont les thèmes sont basés sur Liquid — un moteur de templates serveur.
- Les sites WordPress avec leurs thèmes PHP et leurs pages générées dynamiquement.
- Les sites HTML statiques — landing pages, sites vitrines, documentation sans build step.
- Les pages injectées dans des CMS où le développeur n'a accès qu'à l'HTML et à un champ script.

Dans tous ces contextes, il est impossible d'utiliser `useAgentTool()` (React), `useAgentTool()` (Vue) ou `use:agentTool` (Svelte). Il n'existe pas de cycle de vie de composant, pas de Provider, pas de store réactif. Le développeur a uniquement accès au DOM natif et à du JavaScript classique.

1.2 La solution — @domos/browser

@domos/browser est le SDK DomOS universel pour tous les environnements web sans framework frontend. Il expose le protocole ADTP complet, le système de tools, le Shadow Context, le HITL et la session persistance via une API impérative JavaScript pure, sans dépendance framework.

Il est distribué en deux formats complémentaires :

- Script CDN (IIFE) — chargeable via une simple balise `<script src=>` dans n'importe quelle page HTML, sans build step, sans npm, sans configuration.
- Module ESM — importable via `import` dans un fichier JavaScript moderne, compatible avec les bundlers (Vite, Webpack, esbuild) ou les modules ES natifs du navigateur.

Le rendu du Widget DomOS et des modaux HITL est assuré par Preact en interne — une bibliothèque de 3KB qui permet un rendu UI léger sans imposer de dépendance au projet hôte. Le développeur n'interagit jamais directement avec Preact — c'est un détail d'implémentation interne du SDK.

1.3 Plateformes cibles prioritaires

Plateforme	Contexte technique	Cas d'usage DomOS typique
Laravel / PHP pur	Templates Blade (Laravel) ou PHP natif — HTML généré côté serveur, JS vanilla en complément	Agent qui pilote des formulaires complexes, guide l'utilisateur dans un parcours métier, répond aux questions sur le contenu de la page
Shopify (Liquid)	Thèmes Liquid — HTML/CSS/JS injecté dans les templates de boutique. Accès	Agent e-commerce qui ajoute au panier, guide la sélection de produits, répond aux questions sur les stocks et variantes

Plateforme	Contexte technique	Cas d'usage DomOS typique
	limité au <head> et aux sections de thème	
WordPress	Thèmes PHP + hooks wp_enqueue_scripts. Pages générées dynamiquement, widgets PHP. WooCommerce pour l'e-commerce	Agent qui répond aux questions sur les articles de blog, guide les achats WooCommerce, personnalise l'expérience de navigation
HTML statique	Fichiers HTML purs, hébergés sur GitHub Pages, Netlify, un serveur Apache/Nginx. Aucun build step	Agent intégré à une landing page, documentation interactive, site vitrine avec assistant de qualification de leads

2. Acteurs et scénarios d'utilisation

2.1 Acteurs

Acteur	Profil	Interaction avec @domos/browser
Développeur intégrateur web	Développeur PHP/Laravel/WordPress qui ne maîtrise pas les frameworks JS modernes	Intègre @domos/browser via CDN ou ESM, déclare les tools via API impérative ou attributs HTML data-domos-*
Développeur Shopify	Développeur de thèmes Liquid, accès limité au JS et au DOM des pages boutique	Injecte le script via theme.liquid, déclare les tools Shopify (panier, produits, variantes) via data attributes
Utilisateur final	Client d'une boutique Shopify, visiteur d'un site Laravel, lecteur d'un blog WordPress	Interagit avec l'agent via le Widget DomOS (texte ou vocal), voit les modaux HITL pour les actions critiques
Agent IA (LLM)	Modèle de langage orchestré par le serveur DomOS	Découvre les tools déclarés dans la page via CONTEXT_UPDATE, invoque les handlers qui manipulent le DOM natif

2.2 CU-B01 — Intégration CDN sur une page HTML statique

Champ	Détail
Identifiant	CU-B01
Nom	Intégration @domos/browser via CDN sur un site HTML statique
Acteur	Développeur intégrateur web
Déclencheur	Le développeur ajoute une balise <script> dans son HTML
Précondition	Serveur DomOS actif ou endpoint DomOS Cloud Pro disponible
Résultat attendu	Agent opérationnel sur la page, Widget visible, tools enregistrés

Exemple d'intégration HTML statique

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <script src="https://cdn.domos.dev/browser@1.0.0/domos.min.js"></script>
</head>
<body>
  <h1 id="page-title">Bienvenue sur notre boutique</h1>
  <button id="btn-contact">Nous contacter</button>

  <!-- Déclaration de tools via attributs HTML -->
  <div data-domos-tool="show_contact_form">
```

```

    data-domos-description="Afficher le formulaire de contact"
    data-domos-risk="none">
</div>

<script>
  DomOS.init({
    apiKey: 'pk_live_xxx',
    endpoint: 'wss://cloud.domos.dev/domos',
    widget: { agentName: 'Alex', mode: 'text' }
  });

  // Tool déclaré via API impérative
  DomOS.registerTool('highlight_section', {
    description: 'Mettre en évidence une section de la page',
    schema: { sectionId: 'string' },
    risk: 'none',
    handler: ({ sectionId }) => {
      document.getElementById(sectionId)
        .style.backgroundColor = '#fffde7';
      return { result: 'Section mise en évidence' };
    }
  });
</script>
</body>
</html>

```

2.3 CU-B02 — Intégration Shopify (Liquid)

Champ	Détail
Identifiant	CU-B02
Nom	Intégration @domos/browser dans un thème Shopify
Acteur	Développeur de thème Shopify
Déclencheur	Injection du script dans theme.liquid + déclaration des tools produit
Précondition	Thème Shopify actif, endpoint DomOS Cloud Pro configuré
Résultat attendu	Agent e-commerce capable d'ajouter au panier, de répondre sur les produits, de guider l'achat

Exemple d'intégration Shopify Liquid

```

{% comment -%} theme.liquid - injection dans le <head> {% endcomment -%}
<script src="https://cdn.domos.dev/browser@1.0.0/domos.min.js"></script>

{% comment -%} product.liquid - page produit {% endcomment -%}
<script>
  DomOS.init({
    apiKey: '{{ shop.metafields.domos.api_key }}',
    endpoint: 'wss://cloud.domos.dev/domos',
    context: {
      page: 'product',

```

```

        productId: '{{ product.id }}',
        productTitle: '{{ product.title }}',
        price: {{ product.price | money_without_currency }},
        available: {{ product.available }},
        variants: {{ product.variants | json }}
    }
});

DomOS.registerTool('add_to_cart', {
    description: 'Ajouter ce produit au panier',
    schema: { variantId: 'string', quantity: 'number' },
    risk: 'low',
    handler: async ({ variantId, quantity }) => {
        await fetch('/cart/add.js', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ id: variantId, quantity })
        });
        return { result: `${quantity} article(s) ajouté(s) au panier` };
    }
});

DomOS.registerTool('select_variant', {
    description: 'Sélectionner une variante du produit (taille, couleur)',
    schema: { variantId: 'string' },
    risk: 'none',
    handler: ({ variantId }) => {
        document.querySelector(`[data-variant-id='${variantId}']`).click();
        return { result: 'Variante sélectionnée' };
    }
});
</script>

```

2.4 CU-B03 — Intégration Laravel (Blade)

Champ	Détail
Identifiant	CU-B03
Nom	Intégration @domos/browser dans un projet Laravel avec Blade
Acteur	Développeur Laravel
Déclencheur	Ajout du script dans le layout Blade principal + déclaration des tools métier
Précondition	Application Laravel en production, endpoint DomOS configuré
Résultat attendu	Agent capable de piloter les formulaires, naviguer entre les pages et répondre aux questions métier

Exemple d'intégration Laravel Blade

```

{{-- layouts/app.blade.php --}}
<script src="https://cdn.domos.dev/browser@1.0.0/domos.min.js"></script>

```

```

{{-- resources/views/commandes/create.blade.php --}}
<script>
  DomOS.init({
    apiKey: '{{ config("domos.api_key") }}',
    endpoint: '{{ config("domos.endpoint") }}',
    context: {
      page: 'create_order',
      userId: {{ auth()->id() }},
      cartTotal: {{ $cartTotal }},
      products: @json($products)
    }
  });

  DomOS.registerTool('fill_form_field', {
    description: 'Remplir un champ du formulaire de commande',
    schema: { fieldName: 'string', value: 'string' },
    risk: 'none',
    handler: ({ fieldName, value }) => {
      const field = document.querySelector(`[name='${fieldName}']`);
      if (!field) return { result: 'Champ non trouvé', status: 'error' };
      field.value = value;
      field.dispatchEvent(new Event('input', { bubbles: true }));
      return { result: `Champ ${fieldName} rempli` };
    }
  });

  DomOS.registerTool('submit_order', {
    description: 'Soumettre la commande après validation',
    schema: {},
    risk: 'critical',
    handler: () => {
      document.getElementById('order-form').submit();
      return { result: 'Commande soumise' };
    }
  });
</script>

```

2.5 CU-B04 — Persistance de session lors d'un rechargement de page

Champ	Détail
Identifiant	CU-B04
Nom	Reprise transparente de la conversation après rechargement de page
Acteur	Utilisateur final, @domos/browser, Serveur DomOS
Déclencheur	L'utilisateur recharge la page (F5, navigation, lien interne)
Précondition	Session active existante avec historique de conversation
Résultat attendu	L'agent reprend la conversation là où elle s'était arrêtée, sans demander à l'utilisateur de se représenter

Flux de persistance

1. L'utilisateur a une conversation active avec l'agent sur la page /produit/123.
2. Il clique sur un lien vers /produit/456 (rechargement complet de la page).
3. Avant la navigation, @domos/browser sauvegarde automatiquement dans localStorage : sessionId, historique des 10 derniers messages, Shadow Context courant.
4. La nouvelle page /produit/456 se charge. DomOS.init() est appelé.
5. @domos/browser détecte un sessionId en localStorage et tente une reconnexion avec ce sessionId.
6. Le serveur DomOS retrouve la session en Redis (si TTL non dépassé) et retourne le HANDSHAKE_ACK avec l'historique.
7. L'agent reprend la conversation. Il sait que l'utilisateur regardait /produit/123 et vient de naviguer vers /produit/456.
8. Un CONTEXT_UPDATE est envoyé avec les nouveaux tools de la page /produit/456.
9. L'agent peut dire : « Je vois que vous êtes passé sur ce nouveau produit — souhaitez-vous comparer avec le précédent ? »

[i] Si le TTL Redis est dépassé (session expirée), une nouvelle session est créée automatiquement. DomosAgent peut néanmoins retrouver le contexte via sa mémoire persistante (si activée).

2.6 CU-B05 — Auto-discovery des tools via attributs HTML

Champ	Détail
Identifiant	CU-B05
Nom	Déclaration de tools sans JavaScript via attributs data-domos-*
Acteur	Développeur intégrateur (PHP/Laravel/WordPress sans JS avancé)
Déclencheur	Présence d'attributs data-domos-tool dans le HTML de la page
Précondition	Script @domos/browser chargé et DomOS.init() appelé
Résultat attendu	Les tools sont automatiquement enregistrés sans code JavaScript supplémentaire

Exemple WordPress — déclaration 100% HTML

```
<!-- WordPress : dans un bloc HTML ou un widget -->
<button
  data-domos-tool="scroll_to_section"
  data-domos-description="Faire défiler vers une section de la page"
  data-domos-risk="none"
  data-domos-schema='{ "sectionId": "string" }'
  data-domos-action="scrollIntoView"
  data-domos-target="#section-{sectionId}"
>
  Aller à la section
</button>
```

```
<div
  data-domos-tool="open_popup"
  data-domos-description="Ouvrir une popup d'information"
  data-domos-risk="none"
  data-domos-schema='{ "popupId": "string" }'
  data-domos-action="show"
  data-domos-target="#popup-{popupId}"
>
</div>
```

[i] Le mécanisme `data-domos-action` supporte des actions DOM prédéfinies (`scrollIntoView`, `show`, `hide`, `click`, `setValue`, `addClass`, `removeClass`) sans code JavaScript. Pour les logiques complexes, l'API impérative `DomOS.registerTool()` reste nécessaire.

3. Fonctionnalités

3.1 Vue d'ensemble des fonctionnalités

Fonctionnalité	Description	Priorité	Version
DomOS.init()	Initialisation du client — connexion WebSocket, handshake ADTP, configuration Widget	Critique	v1.0
DomOS.registerTool()	Déclaration impérative d'un tool avec description, schéma, niveau de risque et handler	Critique	v1.0
Auto-discovery data-domos-*	Scan automatique du DOM pour enregistrer les tools déclarés via attributs HTML	Critique	v1.0
Shadow Context	DomOS.setContext() — injection de données contextuelles vers l'agent sans framework	Critique	v1.0
Session Persistence	Sauvegarde automatique sessionId + historique en localStorage, reprise transparente au rechargement	Critique	v1.0
DomOSWidget intégré	Widget chat flottant autonome (Preact) injecté automatiquement dans le DOM de la page hôte	Critique	v1.0
HITL natif	Modaux d'approbation Shadow DOM pour les tools à risque high/critical — même comportement que React/Vue/Svelte	Critique	v1.0
Multi-page persistence	Maintien de la session lors des navigations multi-pages (MPA) — pas seulement les SPAs	Haute	v1.0
DomOS.sendText()	API impérative pour envoyer un message texte à l'agent depuis n'importe quel événement JS	Haute	v1.0
DomOS.onResponse()	Callback appelé à chaque réponse de l'agent — pour affichage custom sans Widget	Haute	v1.0
DomOS.unregisterTool()	Désengagement d'un tool (ex: quand un élément DOM est retiré de la page)	Moyenne	v1.0
DomOS.updateContext()	Mise à jour partielle du Shadow Context sans réinitialisation	Moyenne	v1.0
Mode vocal	Support audio PCM via DomOS.startVoice() / DomOS.stopVoice() — même que les SDKs framework	Moyenne	v1.1

Fonctionnalité	Description	Priorité	Version
DomosAgent mémoire	Intégration DomosAgent pour mémoire persistante cross-session via DomOS Cloud Pro	Haute	v1.1

3.2 API publique de @domos/browser

3.2.1 DomOS.init(config)

Point d'entrée unique du SDK. Initialise la connexion WebSocket, configure le Widget, lance le scan auto-discovery et active la session persistance.

Paramètre	Type	Obligatoire	Description
apiKey	string	Oui	Clé API client DomOS (format pk_live_* ou pk_dev_*)
endpoint	string	Oui	URL WebSocket du serveur DomOS (wss://... ou ws://localhost)
context	object	Non	Shadow Context initial — données de la page à injecter dans l'agent
widget	WidgetConfig	Non	Configuration du Widget DomOS (voir section Widget). Si absent, Widget non affiché.
session	SessionConfig	Non	Configuration de la persistance de session (TTL, storageKey, autoResume)
autoDiscover	boolean	Non (défaut: true)	Active le scan automatique des attributs data-domos-tool dans le DOM
onReady	() => void	Non	Callback appelé quand la connexion est établie et le handshake complété
onError	(err) => void	Non	Callback appelé en cas d'erreur de connexion ou de protocole

3.2.2 DomOS.registerTool(name, definition)

Paramètre	Type	Obligatoire	Description
name	string	Oui	Nom unique du tool — identique à ce que le LLM appellera
definition.description	string	Oui	Description lisible par le LLM — explique ce que fait le tool
definition.schema	object (JSON Schema)	Oui	Paramètres attendus — JSON Schema standard (pas Zod — pas de build step requis)

Paramètre	Type	Obligatoire	Description
definition.risk	none low high critical	Non (défaut: none)	Niveau de risque HITL
definition.handler	async (args) => result	Oui	Fonction exécutée quand le LLM invoque ce tool

3.2.3 Autres méthodes de l'API publique

Méthode	Signature	Description
DomOS.setContext()	setContext(data: object): void	Remplace le Shadow Context complet et envoie un CONTEXT_UPDATE
DomOS.updateContext()	updateContext(patch: object): void	Merge partiel du Shadow Context — seules les clés fournies sont mises à jour
DomOS.sendText()	sendText(message: string): void	Envoie un message texte à l'agent — déclenche le pipeline LLM
DomOS.onResponse()	onResponse(cb: (text, done) => void): void	S'abonne aux réponses de l'agent — pour affichage custom sans Widget
DomOS.onToolCall()	onToolCall(cb: (name, args) => void): void	Callback appelé avant chaque exécution de tool — pour logging ou analytics custom
DomOS.unregisterTool()	unregisterTool(name: string): void	Retire un tool du registry — envoie CONTEXT_UPDATE mis à jour
DomOS.startVoice()	startVoice(): Promise<void>	Active le mode vocal — demande accès micro, démarre le streaming audio
DomOS.stopVoice()	stopVoice(): void	Arrête le mode vocal et le streaming audio
DomOS.disconnect()	disconnect(): void	Ferme proprement la connexion WebSocket
DomOS.getSession()	getSession(): SessionInfo	Retourne les infos de session courante (sessionId, status, tokenCount, etc.)

4. Auto-discovery — Attributs data-domos-*

4.1 Principe

L'auto-discovery permet de déclarer des tools DomOS directement dans le HTML, sans écrire de JavaScript. C'est particulièrement utile pour les développeurs PHP/Laravel/WordPress qui préfèrent travailler dans leurs templates plutôt que dans des fichiers JS séparés.

Au démarrage (DomOS.init()) avec autoDiscover: true), le SDK scanne le DOM à la recherche d'éléments portant l'attribut data-domos-tool. Pour chaque élément trouvé, un tool est automatiquement enregistré. Un MutationObserver surveille ensuite les changements du DOM pour enregistrer ou désengager les tools dynamiquement.

4.2 Attributs disponibles

Attribut	Obligatoire	Description	Exemple
data-domos-tool	Oui	Nom du tool — déclenche l'enregistrement automatique	"add_to_cart"
data-domos-description	Oui	Description du tool pour le LLM	"Ajouter le produit au panier"
data-domos-risk	Non (none)	Niveau de risque HITL	"low"
data-domos-schema	Non ({})	JSON Schema des paramètres attendus (chaîne JSON)	"{\\"qty\\": \\"number\\"}"
data-domos-action	Non	Action DOM prédéfinie à exécuter (sans JS)	"show" "hide" "click" "scrollIntoView" "setValue" "addClass" "removeClass"
data-domos-target	Non (l'élément lui-même)	Sélecteur CSS de l'élément cible de l'action — supporte {param} pour l'interpolation	"#modal-{id}"
data-domos-context	Non	Données JSON à ajouter au Shadow Context quand ce tool est présent	"{\\"productId\\": \\"123\\"}"

4.3 Actions DOM prédéfinies

Les actions prédéfinies (data-domos-action) permettent d'effectuer des manipulations DOM courantes sans écrire de JavaScript. Le SDK les exécute directement en utilisant le sélecteur data-domos-target interpolé avec les arguments du tool call :

Action	Comportement DOM	Paramètres interpolables
show	Retire la classe hidden et définit display: block sur l'élément cible	Paramètres du schema interpolés dans le sélecteur target
hide	Ajoute la classe hidden et définit display: none sur l'élément cible	Paramètres du schema interpolés dans le sélecteur target
click	Appelle .click() sur l'élément cible — déclenche les event listeners existants	Paramètres du schema interpolés dans le sélecteur target
scrollIntoView	Appelle .scrollIntoView({ behavior: 'smooth' }) sur l'élément cible	Paramètres du schema interpolés dans le sélecteur target
setValue	Définit .value sur un input/select/textarea + déclenche l'événement input	Paramètre value requis dans le schema
addClass	Ajoute une ou plusieurs classes CSS sur l'élément cible	Paramètre className requis dans le schema
removeClass	Retire une ou plusieurs classes CSS de l'élément cible	Paramètre className requis dans le schema

[i] Les actions prédéfinies couvrent 80% des cas d'usage simples. Pour les logiques complexes (appels API, calculs, conditions), l'API impérative DomOS.registerTool() est nécessaire et peut coexister avec les attributs HTML.

5. Session Persistence — Architecture complète

5.1 Le problème des MPA (Multi-Page Applications)

Contrairement aux SPAs (Single-Page Applications) où la page ne se recharge jamais, les sites Laravel, Shopify, WordPress et HTML statique sont des MPAs : chaque navigation provoque un rechargement complet de la page et donc une nouvelle exécution de tous les scripts. Sans mécanisme de persistance, chaque rechargement équivaut à une nouvelle session — l'agent perd tout contexte.

5.2 Architecture de persistance — 3 niveaux combinés

Niveau	Mécanisme	Portée	Durée
Niveau 1 — localStorage	Sauvegarde du sessionId + historique récent (10 derniers messages) + Shadow Context dans localStorage avant navigation	Même domaine, même navigateur	Jusqu'à expiration explicite ou TTL (défaut: 30 min)
Niveau 2 — Redis Server-Side	Le serveur DomOS conserve la session complète en Redis avec un TTL étendu pour les sessions browser (défaut: 30 min vs 5 min pour les SPAs)	Toute connexion avec le même sessionId	30 min d'inactivité (configurable)
Niveau 3 — DomosAgent Memory	DomosAgent mémorise le contexte utilisateur cross-session — même après expiration du TTL Redis ou fermeture du navigateur	Cross-session, cross-device (via userId)	Persistance longue durée (Redis + PostgreSQL)

5.3 Flux de reprise de session (Session Resume)

Le flux de session resume est entièrement automatique et transparent pour l'utilisateur :

Étape	Action @domos/browser	Action Serveur DomOS
1. Avant navigation	Sauvegarde dans localStorage: { sessionId, history: [...], context: {...}, savedAt: timestamp }	—
2. Rechargement page	DomOS.init() est appelé avec la nouvelle configuration de la page	—
3. Détection sessionId	Lit localStorage, vérifie que savedAt < TTL (défaut: 30 min)	—
4. HANDSHAKE_INIT avec resume	Envoie HANDSHAKE_INIT avec { sessionId, resumeRequested: true }	Cherche la session en Redis par sessionId
5a. Session trouvée (Redis)	Reçoit HANDSHAKE_ACK avec { sessionId, resumed: true, historyLength: N }	Retourne la session complète avec historique

Étape	Action @domos/browser	Action Serveur DomOS
5b. Session expirée (Redis)	Reçoit HANDSHAKE_ACK avec { sessionId: newId, resumed: false }	Crée une nouvelle session — injecte l'historique localStorage comme contexte initial
6. CONTEXT_UPDATE	Envoie les tools actifs de la nouvelle page et le nouveau Shadow Context	Met à jour le tool registry de la session
7. Continuité agent	L'agent connaît l'historique et les nouveaux tools disponibles	DomosAgent peut enrichir avec la mémoire persistante

5.4 Configuration de la persistance

Option (session config)	Type	Défaut	Description
enabled	boolean	true	Active/désactive la session persistance
storageKey	string	'domos_session'	Clé localStorage utilisée pour sauvegarder la session
ttlMs	number	1800000	TTL de la session locale en ms (défaut: 30 min)
maxHistoryMessages	number	10	Nombre maximum de messages de l'historique sauvegardés localement
autoResume	boolean	true	Tente automatiquement de reprendre la session au chargement
onResume	() => void	undefined	Callback appelé quand une session est reprise avec succès
onNewSession	() => void	undefined	Callback appelé quand une nouvelle session est créée (pas de resume)

5.5 Intégration DomosAgent — Mémoire cross-session

Lorsque DomosAgent est activé (plan Cloud Pro), la persistance va au-delà du TTL Redis. DomosAgent mémorise le profil et le comportement de l'utilisateur de manière persistante :

- Identification de l'utilisateur — si l'application passe un userId dans le Shadow Context, DomosAgent associe la mémoire à cet utilisateur.
- Mémoire de navigation — DomosAgent se souvient des pages visitées, des produits consultés, des intentions exprimées.
- Mémoire des préférences — langue, style de réponse préféré, sujets d'intérêt détectés au fil des conversations.
- Continuité cross-device — si le même userId est détecté sur un autre appareil, DomosAgent retrouve le contexte.

```
DomOS.init({
  apiKey: 'pk_live_xxx',
  endpoint: 'wss://cloud.domos.dev/domos',
  context: {
    userId: '{{ auth()->id() }}', // Laravel - identifie l'utilisateur
    userEmail: '{{ auth()->user()->email }}'
  },
  session: {
    ttlMs: 30 * 60 * 1000, // 30 minutes
    autoResume: true
  }
});
```

6. Architecture technique de @domos/browser

6.1 Stack de build

Outil	Rôle	Justification
Preact	Rendu UI interne (Widget, modaux HITL)	3KB gzippé — compatible Shadow DOM — API React-like — aucune dépendance imposée au projet hôte
esbuild	Bundler pour la production du CDN et du module ESM	Vitesse de build extrême — output minimal — tree-shaking natif — configuration simple
TypeScript	Langage source	Typage fort — cohérence avec les autres packages DomOS — DX améliorée pour les consommateurs ESM
Zod (optionnel)	Validation schema en mode ESM uniquement	Non inclus dans le bundle CDN (trop lourd) — remplacé par JSON Schema validation légère en interne

6.2 Formats de distribution

Format	Fichier	Usage	Taille estimée
IIFE (CDN)	domos.min.js	Chargement via <script src=> — expose window.DomOS globalement	~25KB gzippé (Preact + core ADTP + Widget)
ESM	domos.esm.js	import { DomOS } from '@domos/browser' — compatible bundlers et modules ES natifs	~22KB gzippé (tree-shaking possible)
ESM minimal (core only)	domos.core.esm.js	Sans Widget ni Preact — pour les développeurs qui implémentent leur propre UI	~8KB gzippé

[i] Le bundle CDN inclut Preact (3KB), le client ADTP, le Widget DomOS, le système HITL Shadow DOM, la session persistence et l'auto-discovery. Tout est self-contained — aucune dépendance externe requise sur la page hôte.

6.3 Structure interne du SDK

Module interne	Fichier source	Responsabilité
DomOSBrowserClient	src/client/BrowserClient.ts	Connexion WebSocket ADTP, handshake, routing messages — adapté MPA (pas de framework lifecycle)

Module interne	Fichier source	Responsabilité
ToolRegistry	src/tools/ToolRegistry.ts	Registre des tools — API impérative + auto-discovery — synchronisation CONTEXT_UPDATE
AutoDiscovery	src/tools/AutoDiscovery.ts	Scan DOM data-domos-*, MutationObserver, exécution actions DOM prédéfinies
SessionPersistence	src/session/SessionPersistence.ts	Lecture/écriture localStorage, gestion TTL, logique de resume au HANDSHAKE
ContextManager	src/context/ContextManager.ts	Gestion du Shadow Context — DomOS.setContext() / updateContext()
HITLHandler	src/hitl/HITLHandler.ts	Évaluation risque, création Shadow DOM modal, gestion approbation/refus
BrowserWidget	src/widget/BrowserWidget.tsx	Composant Preact du Widget — injection dans Shadow DOM de la page hôte
BrowserNotification	src/widget/BrowserNotification.tsx	Toast Preact pour les tools de niveau low
EventBus	src/events/EventBus.ts	Bus d'événements interne — onResponse, onToolCall, onError, onReady

6.4 Injection du Widget dans la page hôte

Le Widget @domos/browser est injecté dans la page hôte de manière non-intrusive :

10. Au DomOS.init(), un div conteneur est créé et ajouté à document.body.
11. Un Shadow Root fermé (mode: closed) est attaché à ce div.
12. Preact rend le BrowserWidget dans ce Shadow Root — totalement isolé du CSS et JS de la page hôte.
13. Le Widget écoute l'EventBus interne pour les réponses de l'agent et les changements d'état.
14. Les interactions utilisateur (clic, frappe, micro) appellent DomOS.sendText() et DomOS.startVoice() de manière transparente.

⚠ Le Widget @domos/browser utilise le même Shadow DOM isolé que les versions React/Vue/Svelte. Un script de la page Shopify ou WordPress ne peut pas masquer ou altérer le Widget via JavaScript ou CSS.

6.5 Compatibilité navigateurs

Feature requise	Chrome	Firefox	Safari	Edge	IE
WebSocket	✓ 23+	✓ 11+	✓ 7+	✓ 12+	✗
Shadow DOM (closed)	✓ 53+	✓ 63+	✓ 10+	✓ 79+	✗

Feature requise	Chrome	Firefox	Safari	Edge	IE
localStorage	✓	✓	✓	✓	✓ (limité)
MutationObserver	✓ 18+	✓ 14+	✓ 6+	✓ 12+	✗
ES Modules natifs (ESM)	✓ 61+	✓ 60+	✓ 10.1+	✓ 16+	✗
WebAudio API (vocal)	✓ 35+	✓ 25+	✓ 14.1+	✓ 79+	✗

***i** IE11 non supporté. Pour les projets nécessitant IE11, le mode dégradé sans Widget affiche simplement une interface HTML basique. DomOS redirige les navigateurs anciens vers un fallback graceful.*

7. Guides d'intégration par plateforme

7.1 Guide Shopify

7.1.1 Injection dans theme.liquid

Le script @domos/browser est injecté dans le fichier theme.liquid (layout principal) pour être disponible sur toutes les pages de la boutique. La clé API est stockée dans les métachamps Shopify pour ne pas être exposée dans le code du thème :

```
{% comment %} Ajout dans theme.liquid avant </body> {% endcomment %}
<script src="https://cdn.domos.dev/browser@1.0.0/domos.min.js"></script>
<script>
  DomOS.init({
    apiKey: '{{ shop.metafields.domos.api_key | escape }}',
    endpoint: 'wss://cloud.domos.dev/domos',
    context: {
      shop: '{{ shop.name }}',
      currency: '{{ cart.currency.iso_code }}',
      cartItemCount: {{ cart.item_count }},
      cartTotal: {{ cart.total_price | money_without_currency }},
      customer: {% if customer %}{ id: '{{ customer.id }}' }{% else %}null{%
endif %}
    },
    widget: {
      agentName: '{{ shop.metafields.domos.agent_name | default: "Alex" }}',
      agentTitle: 'Assistant boutique',
      mode: 'text'
    },
    session: { ttlMs: 30 * 60 * 1000, autoResume: true }
  });
</script>
```

7.1.2 Tools globaux Shopify (toutes pages)

```
DomOS.registerTool('get_cart', {
  description: 'Obtenir le contenu du panier',
  schema: {},
  risk: 'none',
  handler: async () => {
    const cart = await fetch('/cart.js').then(r => r.json());
    return { result: cart };
  }
});

DomOS.registerTool('navigate_to', {
  description: 'Naviguer vers une page de la boutique',
  schema: { url: 'string' },
  risk: 'none',
  handler: ({ url }) => {
    window.location.href = url;
    return { result: 'Navigation en cours' };
  }
});
```

7.2 Guide WordPress

7.2.1 Enqueue via functions.php

```
// functions.php
function domos_enqueue_scripts() {
    wp_enqueue_script(
        'domos-browser',
        'https://cdn.domos.dev/browser@1.0.0/domos.min.js',
        [],
        '1.0.0',
        true // Chargement en footer
    );

    wp_add_inline_script('domos-browser', '
    DomOS.init({
        apiKey: "" . get_option('domos_api_key') . "",
        endpoint: "" . get_option('domos_endpoint') . "",
        context: {
            page: "" . get_post_type() . "",
            postId: "" . get_the_ID() . "",
            userId: "" . (is_user_logged_in() ? get_current_user_id() : 'null') . ""
        },
        widget: { agentName: "" . get_option('domos_agent_name', 'Alex') . "" }
    });
    ');
}
add_action('wp_enqueue_scripts', 'domos_enqueue_scripts');
```

7.3 Guide HTML statique (GitHub Pages / Netlify)

Intégration minimale — 5 lignes

```
<!-- index.html - intégration complète en 5 lignes -->
<script src="https://cdn.domos.dev/browser@1.0.0/domos.min.js"></script>
<script>
    DomOS.init({
        apiKey: 'pk_live_xxx',
        endpoint: 'wss://cloud.domos.dev/domos',
        widget: { agentName: 'Alex', mode: 'text' }
    });
</script>

<!-- Tools déclarés 100% en HTML - zéro JavaScript supplémentaire -->
<section
    data-domos-tool="show_pricing"
    data-domos-description="Afficher la section tarifs"
    data-domos-action="scrollIntoView"
    data-domos-target="#pricing">
</section>
```

8. Cas d'erreur et comportements attendus

Code	Scénario	Comportement @domos/browser
ERR-B01	Script CDN non chargé (réseau indisponible)	window.DomOS est undefined — la page continue de fonctionner normalement sans agent. Aucune erreur bloquante.
ERR-B02	Clé API invalide	Le serveur rejette la connexion WebSocket. Le Widget affiche l'état error. Aucun tool n'est exécuté. Callback onError appelé.
ERR-B03	Rechargement avec sessionId expiré (TTL Redis dépassé)	Nouvelle session créée. L'historique localStorage est injecté comme contexte initial si disponible. DomosAgent reprend depuis la mémoire persistante si activé.
ERR-B04	localStorage indisponible (mode privé Safari)	Session persistence désactivée silencieusement. Chaque chargement de page crée une nouvelle session. Un warning console est émis.
ERR-B05	data-domos-target pointe vers un élément inexistant	TOOL_RESULT retourné avec status: error et message descriptif. L'agent informe l'utilisateur que l'élément est absent.
ERR-B06	Conflit de nom de tool (registerTool sur un nom existant)	Un warning console est émis. Le nouveau tool remplace l'ancien. Comportement explicite et prévisible.
ERR-B07	Rechargement rapide avant sauvegarde localStorage	@domos/browser utilise le beforeunload event pour garantir la sauvegarde synchrone avant navigation.
ERR-B08	Accès micro refusé (mode vocal)	Fallback automatique vers le mode texte si widget.fallbackToText = true (défaut). Notification affichée à l'utilisateur.

9. Exigences

9.1 Exigences fonctionnelles

ID	Exigence	Priorité
EF-B01	DomOS.init() doit fonctionner sans aucune dépendance npm — chargeable via un simple tag <script> CDN.	Critique
EF-B02	DomOS.registerTool() doit accepter un JSON Schema standard (pas Zod) pour la définition des paramètres.	Critique
EF-B03	L'auto-discovery data-domos-* doit scanner le DOM au init() et suivre les modifications via MutationObserver.	Critique
EF-B04	La session persistence doit sauvegarder le sessionId avant toute navigation (beforeunload event) et tenter un resume automatique au prochain DomOS.init().	Critique
EF-B05	Le Widget DomOS doit être rendu dans un Shadow DOM isolé — le CSS et JS de la page hôte ne doivent pas l'affecter.	Critique
EF-B06	Le système HITL (4 niveaux de risque, Shadow DOM modal) doit fonctionner de manière identique aux SDKs framework.	Critique
EF-B07	Le bundle CDN (IIFE) ne doit pas dépasser 30KB gzippé.	Haute
EF-B08	DomOS.onResponse() doit permettre un affichage custom des réponses de l'agent sans utiliser le Widget intégré.	Haute
EF-B09	Les actions DOM prédéfinies (show, hide, click, scrollIntoView, setValue, addClass, removeClass) doivent être disponibles via data-domos-action.	Haute
EF-B10	Le SDK doit fonctionner sans erreur sur les 4 plateformes cibles (Laravel, Shopify, WordPress, HTML statique).	Critique

9.2 Exigences techniques

ID	Exigence	Priorité
ET-B01	Le build du SDK doit produire les deux formats simultanément (IIFE CDN + ESM) via une seule commande esbuild.	Critique
ET-B02	Preact est utilisé uniquement en interne pour le rendu du Widget et des modaux. Il ne doit pas être exposé dans l'API publique.	Critique
ET-B03	Le DomOSBrowserClient doit réutiliser les types ADTP de @domos/core — pas de redéfinition des interfaces.	Haute
ET-B04	La session persistence doit être testable sans navigateur (mock localStorage pour les tests unitaires Jest/jsdom).	Haute
ET-B05	L'auto-discovery doit utiliser un MutationObserver unique (pas un par élément) pour limiter l'impact mémoire.	Haute

ID	Exigence	Priorité
ET-B06	Le bundle CDN doit être disponible sur un CDN géo-distribué (Cloudflare CDN) avec un header Cache-Control long (1 an) et versioning dans l'URL.	Haute
ET-B07	Le package @domos/browser doit être publié sur npm avec exports map pour les consommateurs ESM modernes.	Moyenne

10. Roadmap et annexes

10.1 Roadmap @domos/browser

Version	Features	Dépendances
v1.0	DomOS.init(), registerTool(), auto-discovery data-domos-*, session persistence (Niveau 1 + 2), Widget intégré Preact, HITL Shadow DOM, API impérative complète	DomOS Server v1.0, @domos/core v1.0
v1.1	Mode vocal (DomOS.startVoice()), DomosAgent mémoire persistante (Niveau 3), SSR compatibility (Next.js / Nuxt), plugin WordPress officiel	DomosAgent v1.0, Redis session persistence
v1.2	Plugin Shopify officiel (app Shopify), intégration WooCommerce native, SDK Webflow, analytics browser (temps sur page, scroll depth)	DomOS Cloud Pro stable
v2.0	Support offline (Service Worker), sync différé des tool calls hors-ligne, intégration Progressive Web App (PWA)	Architecture offline-first

10.2 Comparaison @domos/browser vs SDKs framework

Feature	@domos/react/vue/svelte	@domos/browser
Déclaration tools	useAgentTool() / use:agentTool (hooks/actions)	DomOS.registerTool() + data-domos-* attributes
Cycle de vie tools	Auto (montage/démontage composant)	Manuel (registerTool / unregisterTool) + MutationObserver
Shadow Context	useAgentContext() / use:agentContext	DomOS.setContext() / DomOS.updateContext()
Widget	Composant framework (DomOSWidget)	Injecté automatiquement dans le DOM (Preact + Shadow DOM)
Session persistence	SPA — pas de rechargement, persistance naturelle	MPA — localStorage + Redis resume
Schema validation	Zod (TypeScript)	JSON Schema (compatible vanilla JS)
Installation	npm install @domos/react	<script src=CDN> ou npm install @domos/browser
Build step requis	Oui (Vite, Webpack, etc.)	Non pour CDN — Oui pour ESM avec bundler
Plateformes cibles	React/Vue/Svelte apps, SPAs, Next.js, Nuxt	Laravel, PHP, Shopify, WordPress, HTML statique, toute MPA

10.3 Références

- CdC Technique général (Doc 2) — Architecture SDK, protocole ADTP
- CdC Fonctionnel (Doc 1) — Scénarios généraux DomOS, système HITL
- WIDGET.md — Configuration WidgetConfig, WidgetTheme, WidgetLabels
- ADTP_PROTOCOL.md — Spécification complète du protocole de communication
- DomosAgent.md — Architecture mémoire persistante (niveau 3 de la session persistence)
- Preact Documentation — <https://preactjs.com>
- esbuild Documentation — <https://esbuild.github.io>
- Shopify Liquid Documentation — <https://shopify.dev/docs/api/liquid>
- WordPress Plugin API — <https://developer.wordpress.org/plugins>